An Open-source LLM-Assisted System for IT/SRE Triage

Kalhar Pandya
Khoury College of Computer Sciences
Northeastern University
Vancouver, Canada
pandya.kal@northeastern.edu

Dawood Sajjadi
Fortinet
Consultant

Maryam Tanha
Khoury College of Computer Sciences
Northeastern University
Vancouver, Canada
m.tanha@northeastern.edu

***Abstract*—During a production incident, Site Reliability Engineering teams must SSH into multiple hosts, correlate logs, and reconcile metrics across monitoring stacks under time pressure. Large Language Models can accelerate parts of this workflow, yet prior work shows they struggle with root cause analysis even when given topology and telemetry data; a single model lacks the structural scaffolding to reason across it. We present an open-source, hierarchical multi-agent framework built on LangGraph that provides this scaffolding through structured decomposition. A Parent Agent drives iterative investigation cycles, delegating subtasks to parallel Specialist Agents (ReAct-based workers that run read-only commands over Docker exec or SSH) while a Synthesis Agent consolidates cross-domain findings between cycles. Operators declare their service configuration in YAML files validated by Pydantic at startup; adding a new specialist requires only a YAML config, no Python. A two-layer security model (command allowlist plus output redaction) prevents agents from modifying infrastructure or leaking credentials. We evaluate on a five-service mail application deployed on an Amazon Web Services (AWS) Elastic Compute Cloud (EC2) instance, injecting six faults across six infrastructure domains. The agent correctly identifies the root cause in all six cases, averaging 89 seconds and $0.27 per investigation on GPT-4.1.**

***Index Terms*—multi-agent systems, large language models, root cause analysis, AIOps, site reliability engineering, LangGraph**

I. Introduction

When a cloud-native microservice stack fails, the response is inherently manual. Site Reliability Engineering (SRE) and DevOps teams open Secure Shell (`SSH`) sessions to multiple hosts, correlate log fragments across services that may share nothing but a request ID, and attempt to isolate root causes from downstream symptoms under time pressure. Surveys and industrial experience confirm that this process remains laborious and error-prone [1], [2]. Large Language Models (LLMs) are increasingly applied to parts of this process, though not as replacements for existing toolchains. A recent AIOps survey [1] recommends integrating LLMs with established monitoring stacks rather than building from scratch, and a chaos-engineering evaluation [3] finds that models function best as co-pilots alongside human SREs. One finding in particular shapes our architecture: even when provided with system topology and oracle telemetry (i.e., ground-truth telemetry with no noise or missing data), the best-performing LLM solved only 5.37% of root cause analysis tasks in the OpenRCA benchmark [4], Without additional structural support, models cannot reason across complex service dependency graphs. The core limitation is asking a single model to process the full topology at once.

We address this through multi-agent decomposition over a declared service topology: rather than asking one model to reason over the entire stack, we break the problem into domain-specific subtasks, each grounded in a narrow slice of the topology, iterate across investigation cycles, and synthesize findings across agents. The framework grounds each specialist in a YAML service topology and enforces a read-only execution model throughout the investigation [5]. Before each specialist begins, the framework pre-fetches container logs and runs configured context commands, injecting structured evidence into the agent's opening message rather than forwarding raw streams that would exhaust a context window. Structured log representations improve LLM parsing accuracy [6], motivating this pre-processing step.

Our contributions are as follows:

- We design and implement an open-source, hierarchical multi-agent framework for automated root cause analysis, using a Parent–Specialist–Synthesis architecture on LangGraph that decomposes investigations into parallel, domain-specific subtasks.
- We introduce a YAML-based profile system that allows operators to describe deployment targets and add specialist agents without writing Python code.
- We implement a two-layer security model combining a command allowlist with output redaction to enforce read-only agent execution and prevent credential exposure.
- We evaluate the framework on a five-service mail application with six injected faults across six infrastructure domains, achieving correct diagnosis in all cases at an average of 89 seconds and $0.27 per run.

The source code is available at https://github.com/maryam-tanha/LLM-assisted-Triage.

II. Related Work

A. Multi-Agent Architectures for IT Operations

Splitting a complex task across specialized agents outperforms asking a single model to handle everything. AgentOrchestra [7] formalizes this with the TEA (Tool-Environment-

Agent) protocol, where a central planner delegates to domain-specific sub-agents; the architecture achieves 89.04% average across difficulty levels on the General AI Assistants (GAIA) benchmark, well above monolithic baselines. MetaGPT [8] assigns each agent a Standardized Operating Procedure (SOP) modeled on real-world software team workflows, which constrains output format and reduces errors during collaboration. In both cases, enforcing role boundaries proved as important as model capability.

Context management remains a recurring difficulty in multi-agent IT systems. An AI-Oriented Operations (AOI) system is proposed in [9]. It compresses intermediate findings before passing them to a high-level observer, retaining 92.8% of critical information at 72.4% compression, while cutting mean time to resolution by 34.4% over the best prior baseline.

Three additional systems inform our design. Triangle [10] demonstrated over 20% triage accuracy improvement (reaching 91.7% accuracy within five reassignment hops) in a production deployment serving tens of millions of users. MasRouter [11] routes queries to appropriately sized models and collaboration configurations when complexity permits, reducing HumanEval overhead by up to 52%. SageCopilot [12] integrates reflective validation and human-in-the-loop feedback; its iterative post-generation correction yielded measurable quality gains.

B. LLM-Based Root Cause Analysis

Feeding raw diagnostic data directly to an LLM yields low accuracy in production settings. RCACopilot at Microsoft [2] addressed this by summarizing structured diagnostic information before passing it to the prediction model, gaining 0.077 absolute Micro-F1 points over the unsummarized baseline. FLASH [13] extends this by decomposing the full diagnostic lifecycle into four supervised states and learning from historical failure runs; across 250 real incidents it reached 73.9% accuracy, and in a separate human evaluation brought average diagnosis time down to 5.3 minutes against a 90-minute mitigation baseline.

Separating agents by function also affects accuracy. MA-RCA [14] splits retrieval and validation into distinct agents and reports 95.8% accuracy on the Nezha cloud-native dataset, a result the authors attribute directly to eliminating context-switching between hypothesis generation and evidence checking. D-Bot [15] applies tree-search reasoning using Upper Confidence bound applied to Trees (UCT) scoring to database anomalies, forcing the planner to explore multiple diagnostic paths rather than committing to a single chain of actions.

Not every scenario suits the same approach. Hy-LIFT [16] shows that for 5G/6G networks, rule-based labeling handles known fault classes well; the LLM layer is reserved for novel or ambiguous cases, reaching a macro-F1 of 0.89. At the other end, an Artificial Intelligence for IT Operations (AIOps) reliability study [3] finds that zero-shot LLMs misclassify benign traffic spikes as security incidents.

C. Specialist Agent Design and Data Ingestion

Ashraf and Talavera [17] observe that conflicting decisions arise when agent coordination is not effectively managed, and that assigning specialized roles leads to more comprehensive evaluation.

Once agents have clear roles, the next question is what data each one should receive. Dumping raw infrastructure logs into an LLM context reliably fails: token budgets overflow and important lines get truncated before the model ever sees them. DivLog [6] addresses template extraction specifically by sampling a maximally diverse set of log candidates offline and using them as in-context examples at inference time. Across 16 LogPAI benchmarks it achieves 98.1% parsing accuracy, which makes structured templates a reliable input rather than an aspiration. Together, these two concerns (clear role boundaries that prevent cross-domain interference, and structured data ingestion that keeps each agent's context window focused) form the foundation of our specialist design.

D. Agent Resilience and Security

Huang et al. [18] inject faulty agents into three topology types and measure the damage: hierarchical systems lose only 5.5% accuracy; flat topologies, lacking clear authority, reach decision paralysis at 10.5%; linear chains, where each hop re-transmits errors, degrade by 23.7%.

On the security side, Kholkar and Ahuja [5] demonstrate a "policy-as-prompt" pattern that encodes governance rules into runtime classifiers; their least-privilege framing directly informs our read-only execution policy.

III. Methodology

Fig. 1 shows the full system. We built it as a three-tier LangGraph graph: a Parent Agent that plans and decides, a pool of Specialist Agents that gather evidence in parallel, and a Synthesis Agent that correlates findings across domains before the next cycle. Specialist workers report exclusively through the orchestrator because flat and linear topologies suffer higher accuracy loss under agent failure [18]. Each product is described in its own profile directory of YAML files, validated by Pydantic at load time; pointing the framework at a different environment requires editing configuration, not code.

A. Service Configuration

The goal is to let operators describe their infrastructure once, in a format they already understand, without writing any agent-specific code. The result is a *profile directory* per product, validated by Pydantic at startup:

- `profile.yaml` – product metadata, access method (`docker_exec` or `ssh`), and a `services` block where each entry captures what a healthy instance looks like (`expected_behavior`), what patterns signal trouble (`known_failures`), and which shell commands collect an initial evidence snapshot (`context_commands`).
- `agents/*.yaml` – one file per specialist, each declaring an `agent_type`, a domain-specific `system_prompt`, routing guidance (`when_to_use`, `do_not_use`), optional agent-level `context_commands`, and a `gather_docker_host_context` flag that controls

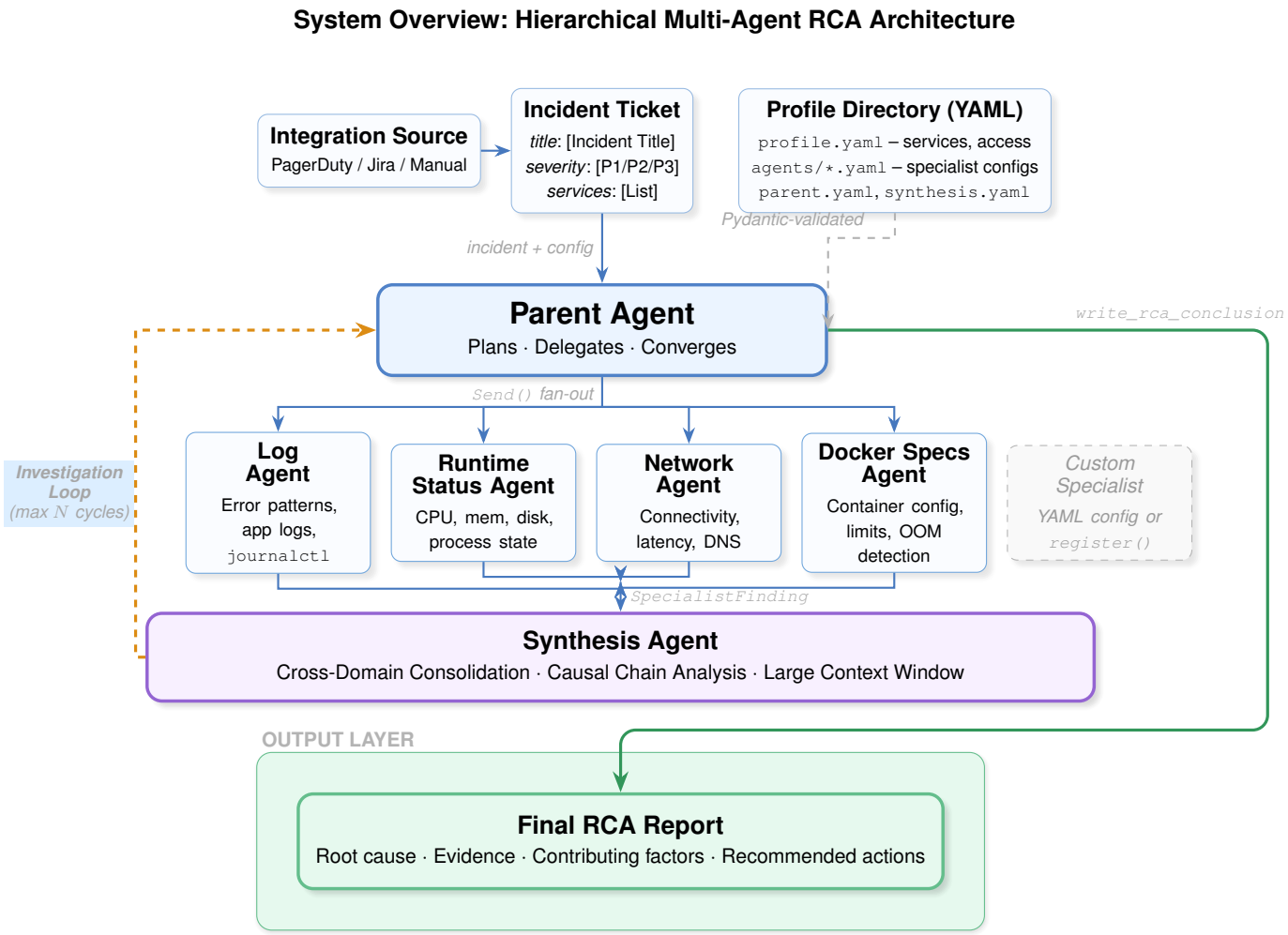

Fig. 1. Multi-agent RCA architecture. The Parent Agent fans out investigation subtasks in parallel via `Send()` and iterates until the accumulated evidence supports a conclusion or the cycle limit is reached. The dashed orange path shows the investigation loop; the green path marks the conclusion route to the final report.

whether Docker-level metadata is collected before the specialist begins.

- `parent.yaml` and `synthesis.yaml` – customizable system prompts for the Parent and Synthesis agents, allowing operators to adjust orchestration behavior per product without modifying framework code.

At runtime the Parent Agent reads the specialist routing hints verbatim when choosing whom to assign. Nothing is hardcoded in the planner prompt; the entire profile is loaded at invocation time, so onboarding a new product amounts to filling in a directory.

Specialists can also be introduced without writing Python at all. A YAML agent configuration file is enough: a generic `YAMLSpecialist` bridge class picks it up at startup, builds the corresponding ReAct agent, and wires the node into the graph. An operator familiar with Kafka internals, for instance, can contribute a specialist by writing a system prompt and a few routing hints; the framework handles registration and graph wiring.

B. Parent Agent and Planning

The Parent Agent is the only node in the graph that can change the investigation's direction. At each cycle it receives the incident summary, the full YAML topology, and the accumulated `cumulative_history` from all prior cycles. We bind the LLM with `tool_choice=``any''` to ensure it always produces a typed action rather than free text. This replanning pattern follows multi-agent RCA designs that explicitly separate retrieval from synthesis [14], [15].

The LLM has exactly two choices. `create_subtasks` emits a list of `Subtask` objects, each specifying a target service, container, natural-language description, a falsifiable hypothesis, and the agent to assign. The LangGraph router converts these into `Send()` fan-out dispatches that run in parallel. `write_rca_conclusion` closes the cycle, producing a structured report with root cause, contributing factors, evidence summary, and recommended actions.

We chose not to rely on the agent voluntarily concluding: once the zero-indexed `current_cycle`

counter reaches MAX_CYCLES (default: 3, yielding up to four investigation rounds), the framework hard-wires write_rca_conclusion as the only permitted tool call; the loop cannot run indefinitely.

Not every incident requires autonomous planning from scratch. The framework therefore supports a *manual investigation mode*: an operator who already suspects a specific service can pre-populate the subtask list before launching the graph. If subtasks exist at cycle 0, the Parent Agent skips the LLM call and fans them out directly. Any subsequent cycles fall back to normal LLM-driven planning, so the autonomous loop remains available even when the first round is hand-directed.

C. Specialist Agents

A specialist is a LangChain Reasoning + Acting (ReAct) agent with a deliberately narrow scope. Its only tool is run_command; its system prompt encodes domain knowledge for one area (logs, runtime resources, network connectivity, container configuration, etc.) and nothing else. Empirically, narrowing the system prompt reduced cross-domain confusions: a log specialist with Redis knowledge in its prompt issued Redis commands during unrelated log investigations.

Before the ReAct loop begins, the framework pre-runs the service's context_commands and, in Docker mode, tails the container logs. Both are injected into the agent's opening message, giving it a concrete evidence baseline without spending any tool-call budget; log output is pre-processed and structured before injection, following evidence that structured templates improve LLM parsing [6]. When the per-agent flag gather_docker_host_context is set, the framework additionally collects docker inspect (full container configuration and state), docker stats --no-stream (a point-in-time Central Processing Unit (CPU), memory, network, and Input/Output (I/O) snapshot), and docker events filtered to the preceding 24 hours (daemon-level signals: Out-of-Memory (OOM) kills, restarts, health-check failures). This host-level collection is necessary because application logs miss certain failure modes entirely: a container killed by the kernel OOM reaper, for example, leaves no application-level trace.

Regardless of how evidence is gathered, the loop runs until the agent emits a structured final answer or hits MAX_ITERATIONS (default: 10), at which point a low-confidence SpecialistFinding is returned so the cycle can continue rather than crash.

Commands run through one of two backends, both presenting the same interface to the agent. *Docker exec* calls docker exec <container> sh -c <cmd> on the local host and is the default for demo deployments. *SSH* uses a pooled paramiko client keyed by (host, port, username), reusing open transport sessions to avoid repeated handshake overhead in production environments.

Adding a specialist requires no changes to the orchestration layer. A new Python module calls register() at import time; the graph builder wires the new node in at startup and the Parent Agent sees it in its routing prompt the next invocation. As noted in Section III-A, a specialist can also be defined purely through YAML: the YAMLSpecialist class reads the config, assembles the prompt and tool bindings, and registers the node, so operators can extend coverage without writing Python.

D. Synthesis

After each cycle's specialists finish, the Synthesis Agent reads new SpecialistFinding objects tracked via a findings_offset pointer to avoid reprocessing prior cycles. Each finding conforms to a fixed typed schema, keeping synthesis predictable. The agent produces a CycleSummary: a narrative paragraph, KEY_FINDINGS bullets, and RECOMMENDATIONS. We give it no tool access deliberately; its job is cross-domain correlation, noticing for example that a memory spike on the Redis container explains the connection-pool exhaustion a log agent reported on the API service. Keeping it tool-free means reasoning over the full evidence window without branching into further data collection. The resulting summary feeds into cumulative_history, which the Parent Agent reads at the start of every subsequent cycle.

Once the Parent Agent calls write_rca_conclusion, the structured report (root cause, contributing factors, evidence summary, and recommended actions) goes directly to the operator.

E. Security Model

We treat agents as untrusted code paths. Every command, regardless of which specialist issues it, passes through two sequential checks before execution and before its output reaches the LLM.

The *command allowlist* applies a deny-first rule: a command is rejected the moment it matches any blocked pattern (destructive operations such as rm and chmod; shell redirection; code injection via | bash; and outbound network tools like curl and nc). Only commands whose prefix appears on the explicit allow-list (log readers, system status utilities, and service Command-Line Interfaces (CLIs) like redis-cli and psql) proceed to execution. Rejected commands return "BLOCKED: <reason>" so the agent can adapt rather than hang.

The *output redactor* scrubs every response before it leaves the execution layer. It strips AWS access keys, PEM private-key blocks, bearer tokens, credential assignments (api_key=, password=), email addresses, full Internet Protocol version 4 (IPv4) addresses (partially preserved for subnet context), and long base64 strings. The redactor runs on every execution path, including the host-side docker logs fetch, so no raw secrets ever appear in the LLM's context.

Both checks run at every node without exception. The least-privilege principle [5] holds across the entire system: agents can read diagnostic state but cannot alter it or exfiltrate credentials.

F. Data Privacy and Provider Governance

Diagnostic data flowing through a live investigation (log excerpts, service configurations, error traces, topology details) is operationally sensitive. Preventing this data from entering model training pipelines is a hard design constraint.

All LLM inference in our framework is routed through the OpenRouter API rather than through direct provider endpoints. OpenRouter is a routing proxy that dispatches each request to a backend model provider but, by default, retains only request metadata (token counts, latency, model identifier) and not the prompt or completion content [19]. Two opt-in flags exist for users who want logging or data sharing, but both are disabled in our deployment.

The more consequential setting is `provider.data_collection`, which can be set to `"deny"` either per-request or at the account level. In our deployment we enable this restriction account-wide, directing OpenRouter to route calls *only* to providers whose terms of service prohibit collecting or training on user data. If no compliant provider is available for the requested model, the call fails rather than silently falling back to one that might retain inputs. A stricter option, `provider.zdr` (zero data retention), is available for deployments that require providers to hold nothing beyond the duration of a single request-response cycle.

The providers we tested against, primarily OpenAI (GPT-4.1) via OpenRouter, independently enforce a no-training default for API traffic. Anthropic, Mistral, and Google apply similar policies on their paid API tiers. These guarantees are contractual rather than technical: routing away from non-compliant providers is the enforcement mechanism, which is standard practice for any API proxy architecture. Operators deploying our framework in regulated environments should verify that their chosen model and provider combination meets their specific compliance requirements, since retention windows and audit-log practices vary across vendors.

IV. Experiments and Results

A. Experimental Setup

We evaluated the framework on a five-service mail application deployed on an AWS EC2 instance (Ubuntu) via Docker Compose. The stack comprises Postfix (Simple Mail Transfer Protocol (SMTP) transport), Dovecot (Internet Message Access Protocol (IMAP)), Roundcube with Apache and PHP (webmail front-end), PostgreSQL (address-book and session storage), and Redis (session cache). This target was chosen over the simpler voting application used during development because it exercises a wider range of infrastructure domains (mail transport, web serving, database connectivity, and Domain Name System (DNS) resolution) within a single deployment.

Load was generated with Locust, configured for 20 concurrent virtual users over 90-second windows. Each Locust user performed a mix of SMTP sends, IMAP logins and mailbox operations, and Roundcube Hypertext Transfer Protocol (HTTP) workflows (login, inbox listing, compose, send). The framework ran with default settings: `MAX_CYCLES`=3, `MAX_ITERATIONS`=10 per specialist, and four specialist agents (Log, Runtime Status, Docker Specs, Network). All inference used OpenAI's GPT-4.1 model via OpenRouter.

We report four metrics per experiment: whether the agent correctly identified the injected root cause (binary accuracy), the number of investigation cycles consumed, wall-clock elapsed time from graph entry to final report, and the estimated cost in US dollars based on OpenRouter per-token pricing.

B. Fault Injection

Each experiment introduces a single-parameter misconfiguration or corruption into one service while leaving the rest of the stack healthy. The faults were designed to mimic realistic operator errors (a connection limit set too low, a memory ceiling that triggers the OOM killer) rather than synthetic crash scenarios. Table I summarizes the six injected faults, spanning six infrastructure domains: mail protocol, database, web server, mail transport, DNS, and OS-level memory.

Every fault was injected via `docker exec` or a host-side command, and a Locust run captured the resulting failure signature before launching the RCA agent. After each experiment the fault was reversed to restore the baseline.

TABLE I
INJECTED FAULTS AND AFFECTED SERVICES.

ID	Domain	Fault	Affected
1	Mail protocol	Dovecot `mail_max_userip_connections=1`	IMAP
2	Database	PostgreSQL `max_connections=4`	Roundcube
3	Web server	PHP `memory_limit=10M`	Roundcube
4	Mail server	Postfix `smtpd_client_message_rate_limit=1`	SMTP
5	DNS	Corrupt `/etc/resolv.conf` in mailserver	Outbound mail
6	OS / Memory	Container hard memory limit 48 MB (OOM)	Roundcube

C. Results

The agent identified the correct root cause in all six experiments. Table II shows the per-experiment breakdown.

TABLE II
RCA AGENT PERFORMANCE ACROSS SIX FAULT INJECTIONS.

ID	Cycles	Time (s)	Cost ($)	Correct
1	4	136.5	0.49	✓
2	1	30.0	0.03	✓
3	2	74.8	0.14	✓
4	1	44.9	0.04	✓
5	3	155.2	0.58	✓
6	2	95.3	0.32	✓
Mean		89.5	0.27	6/6

Investigation times ranged from 30 seconds (EXP-02, a single-cycle diagnosis) to just over two and a half minutes (EXP-05, three cycles). Token costs stayed between $0.03 and $0.58 per run, with an average of $0.27.

D. Analysis

Fault indirection correlates directly with cycle count. Faults that left an unambiguous signature in the affected service's own logs or configuration (the PostgreSQL connection pool in EXP-02, the Postfix rate-limit rejection code in EXP-04) were resolved in a single cycle. The agent read the error, confirmed the configuration, and concluded. Faults whose symptoms surfaced one layer away from the actual cause required more work. In EXP-05, for instance, DNS resolution failures appeared in Postfix delivery logs, but the root cause was a corrupted `/etc/resolv.conf` inside the container. The agent spent its first cycle investigating Postfix configuration (which was clean), broadened its network investigation in cycle 2, and only in cycle 3 checked the resolver file directly. This three-cycle progression exercised the iterative narrowing that the synthesis loop provides.

EXP-06 highlights why host-level container metadata matters. An OOM kill leaves no trace in application logs; the process simply disappears. The Docker Specs specialist, running with `gather_docker_host_context` enabled, retrieved `docker inspect` (which reported `OOMKilled: true` and an incrementing restart count) and `docker stats` (which showed memory pinned at 100% of the 48 MB limit). Without these signals the agent would have been limited to observing intermittent 502 errors with no clear cause.

Breadth-first specialist dispatch also contributed to the results. In every experiment the Parent Agent sent subtasks to all four specialists in cycle 1, even when the incident description pointed toward a single service. Specialists that found nothing still contributed by ruling out competing hypotheses: confirming that Redis, DNS, or host resources were healthy allowed the Synthesis Agent to rank the real finding higher in its hypothesis tier.

At an average of $0.27 per investigation, token cost is negligible relative to the engineering time a manual triage typically consumes. The most expensive run (EXP-05, $0.58) corresponds to the three-cycle investigation; single-cycle cases came in under five cents.

V. Conclusion and Future Directions

We presented an open-source, hierarchical multi-agent framework for automated root cause analysis in microservice environments. The central result is that decomposing an investigation into domain-specific agents, each grounded in a narrow slice of a declared service topology, succeeds where prior single-model approaches have not [4]. Iterative cycles with cross-domain synthesis were necessary for faults whose symptoms surfaced in a different service than the actual cause. The deny-first security model kept agents useful throughout without giving them the ability to modify infrastructure or leak credentials.

The framework's profile system lets operators describe new deployment targets entirely in YAML (services, specialist agents, and orchestrator prompts) without writing Python. On our five-service mail application, six fault injections spanning six infrastructure domains were all diagnosed correctly, averaging 89 seconds and $0.27 per run on GPT-4.1. Direct configuration faults resolved in one cycle; faults whose symptoms appeared in a different service's logs took up to three, exercising exactly the iterative narrowing the architecture was designed for.

Several limitations remain. We tested a single model (GPT-4.1) on one infrastructure target with controlled fault injections, not organic incidents. Different architectures or smaller models may trade accuracy for cost in ways we have not measured. Production environments are noisier, faults compound, and the "correct" root cause is sometimes a matter of judgment rather than a single misconfigured parameter. A broader evaluation covering multiple models, Kubernetes-native deployments, and real incident data would strengthen the claims considerably.

The most immediate next step is an automated evaluator agent that scores draft reports on accuracy, completeness, and actionability, following the review pattern validated by SageCopilot [12]. We also plan to adopt the Model Context Protocol [20] for dynamic tool discovery, so specialists can pick up new monitoring backends without per-integration code. Longer-term, a human-in-the-loop command approval mode would serve environments where full autonomy is not yet appropriate: the agent proposes a command, the operator approves or rejects it, and the framework records the decision for future policy learning.

Acknowledgment

This work was conducted as part of a Research Apprenticeship at Khoury College of Computer Sciences, Northeastern University, under the supervision of Dr. Maryam Tanha. The authors thank Khoury College for providing the AWS and OpenRouter credits used in the experimental evaluation.

References

[1] L. Zhang, T. Jia, M. Jia, Y. Wu, A. Liu, Y. Yang, Z. Wu, X. Hu, P. Yu, and Y. Li, "A survey of AIOps in the era of large language models," *ACM Computing Surveys*, 2025.

[2] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, and J. Zeng, "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the Nineteenth European Conference on Computer Systems*, pp. 674–688, ACM, 2024.

[3] T. Szandała, "AIOps for reliability: Evaluating large language models for automated root cause analysis in chaos engineering," in *International Conference on Computational Science*, (Cham), pp. 323–336, Springer Nature Switzerland, 2025.

[4] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, "OpenRCA: Can large language models locate the root cause of software failures?," in *The Thirteenth International Conference on Learning Representations*, 2025.

[5] G. Kholkar and R. Ahuja, "The AI agent code of conduct: Automated guardrail policy-as-prompt synthesis." arXiv e-prints, 2025.

[6] J. Xu, R. Yang, Y. Huo, C. Zhang, and P. He, "DivLog: Log parsing with prompt enhanced in-context learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–12, ACM, 2024.

[7] W. Zhang, L. Zeng, Y. Xiao, Y. Li, Y. Zhao, C. Cui, Y. Liu, and B. An, "AgentOrchestra: Orchestrating hierarchical multi-agent intelligence with the tool-environment-agent (TEA) protocol." arXiv / Cornell University, 2025.

[8] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, and L. Zhou, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[9] Z. Bai, E. Ge, and J. Hao, "Multi-agent collaborative framework for intelligent IT operations: An AOI system with context-aware compression and dynamic task scheduling," 2025. arXiv preprint arXiv:2512.13956.

[10] Z.-H. Yu, M. Ma, X. Feng, R. Ding, C. Zhang, Z. Li, M. Chintalapati, X. Zhang, R. Wang, C. Bansal, and S. Rajmohan, "Triangle: Empowering incident triage with multi-LLM-agents," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, ACM, 2025.

[11] Y. Yue, G. Zhang, B. Liu, G. Wan, K. Wang, D. Cheng, and Y. Qi, "MasRouter: Learning to route LLMs for multi-agent systems," 2025. arXiv preprint arXiv:2502.11133.

[12] Y. Liao, J. Bian, Y. Yun, S. Wang, Y. Zhang, J. Chu, T. Wang, Y. Li, X. Li, S. Ji, and H. Xiong, "SageCopilot: An LLM-empowered autonomous agent for data science as a service," *IEEE Transactions on Services Computing*, 2025.

[13] X. Zhang, T. Mittal, C. Bansal, R. Wang, M. Ma, Z. Ren, H. Huang, and S. Rajmohan, "FLASH: A workflow automation agent for diagnosing recurring incidents." Microsoft Research, 2024.

[14] F. Fu, H. Ding, Y. Qin, J. Yu, and D. Xu, "Leveraging multi-agent framework for root cause analysis," *Complex & Intelligent Systems*, vol. 12, p. 4, January 2026.

[15] X. Zhou, G. Li, Z. Sun, Z. Liu, W. Chen, J. Wu, J. Liu, R. Feng, and G. Zeng, "D-Bot: Database diagnosis system using large language models," 2023. arXiv preprint arXiv:2312.01454.

[16] A. D. Salman, A. T. Zeyad, S. S. Jumaa, S. M. Raafat, F. H. Jasim, and A. J. Humaidi, "Hybrid LLM-assisted fault diagnosis framework for 5G/6G networks using real-world logs," *Computers*, vol. 14, no. 12, p. 551, 2025.

[17] B. Ashraf and G. Talavera, "Autonomous agents in software engineering: A multi-agent LLM approach." ResearchGate, 2025. Preprint available via ResearchGate.

[18] J.-T. Huang, J. Zhou, T. Jin, X. Zhou, Z. Chen, W. Wang, Y. Yuan, M. R. Lyu, and M. Sap, "On the resilience of LLM-based multi-agent collaboration with faulty agents," 2024. arXiv preprint arXiv:2408.00989.

[19] OpenRouter, "Data collection and privacy." OpenRouter Documentation, 2025. Industry reference; not peer-reviewed.

[20] A. Addepalli, E. Schoppik, J. Cooke, K. Blacutt, M. Khanuja, and N. van der Smagt, "Unlocking the power of model context protocol (MCP) on AWS." AWS Machine Learning Blog, June 2025. Industry reference; not peer-reviewed.